

CRICKET ADMIN BACKEND

Complete Technical Onboarding & Architecture Documentation

Express.js · MongoDB · Socket.IO · Zod · JWT · Docker

Project	Cricbuzz - Backend
Version	1.0.0
Runtime	Node.js 20 (LTS)
Framework	Express.js 5
Database	MongoDB 7 (Mongoose 9)
Real-time	Socket.IO 4
Auth	JWT (jsonwebtoken 9)
Validation	Zod 4
Containerisation	Docker + Docker Compose
Documentation Date	June 2026

Table of Contents

- 1. Project Overview**
- 2. Complete Folder Structure**
- 3. Folder-by-Folder Analysis**
 - 3.1. `server/`
 - 3.2. `src/config/`
 - 3.3. `src/modules/auth/`
 - 3.4. `src/modules/users/`
 - 3.5. `src/modules/series/`
 - 3.6. `src/modules/team/`
 - 3.7. `src/modules/player/`
 - 3.8. `src/modules/squad/`
 - 3.9. `src/modules/match/`
 - 3.10. `src/modules/playing-xi/`
 - 3.11. `src/modules/score/`
 - 3.12. `src/modules/commentary/`
 - 3.13. `src/modules/user/` (Public)
 - 3.14. `src/shared/`
 - 3.15. `src/sockets/`
 - 3.16. `seed/`
- 4. File-by-File Analysis**
- 5. Application Flow**
- 6. Architecture Analysis**
- 7. API Documentation**
- 8. Database Documentation**
- 9. Configuration Files**
- 10. Security Analysis**
- 11. Real-Time (Socket.IO) Architecture**
- 12. Caching Architecture**
- 13. Complete Execution Flow**
- 14. Developer Onboarding Guide**
- 15. Project Summary & Recommendations**

1. Project Overview

1.1 Project Name & Purpose

The project is the **Cricbuzz - Backend** - a production-grade RESTful API server that powers a full-featured cricket management and live-scoring platform. It exposes two distinct surfaces:

- **Admin API** - authenticated, role-protected endpoints consumed by an operator dashboard (super-admins, admins, scorers) to manage series, teams, players, squads, matches, scores, and commentary.
- **Public API** - unauthenticated, heavily-cached GET endpoints consumed by fan-facing web/mobile clients for home feeds, live match centres, scorecards, commentary, series standings, and global search.

1.2 Core Features

Auth & RBAC	JWT-based authentication with three roles: SUPER_ADMIN, ADMIN, SCORER. Every admin endpoint enforces role membership.
Series Management	CRUD for cricket series with season uniqueness, soft-delete protection (cannot delete if matches exist).
Team & Squad	Team CRUD with logo/colour, squad management (add/remove players with idempotency via \$addToSet / \$pull).
Player Registry	Global player catalogue with batting/bowling styles, soft-delete.
Match Lifecycle	Strict state-machine: UPCOMING → TOSS_COMPLETED → PLAYING_XI_SELECTED → LIVE → INNINGS_BREAK → COMPLETED.
Playing XI Selection	Validates exactly 11 players, no duplicates, one captain, one wicket-keeper, all from registered squad.
Live Scoring	Real-time score creation/update during LIVE matches, with run-rate and target tracking.
Commentary	Ball-by-ball commentary (NORMAL/FOUR/SIX/WICKET/MILESTONE) with paginated public feed.
Real-Time Push	Socket.IO rooms per match. Score, commentary, toss, playing-XI, start, and complete events broadcast instantly.
Points Table	Derived on-the-fly from completed matches: played/won/lost/points sorted by points desc, won desc.
Global Search	Cross-entity regex search across players, teams, and series (min 2 chars, max 10 results per entity).
Response Cache	In-memory TTL cache for all public GET routes. TTLs: 5–60 seconds depending on data volatility.
Seeder	One-command database seeder with realistic BGT-2026 fixture, players, and live commentary samples.
Docker	Production-ready Dockerfile + docker-compose with MongoDB 7, healthchecks, and named volumes.

1.3 Tech Stack

Runtime	Node.js 20 LTS (Alpine in Docker)
Framework	Express.js 5 (latest stable)
Database	MongoDB 7 via Mongoose ODM 9
Real-time	Socket.IO 4 over native http.Server
Auth	bcryptjs 3 (password hashing), jsonwebtoken 9 (JWT)
Validation	Zod 4 (schema-first, compile-time-safe)
HTTP client	axios 1.6 (used in seed/scrapper utilities)
Scraping	cheerio 1 + puppeteer 25 (data import utilities)
Config	dotenv 17 (env file loading)
Container	Docker (node:20-alpine), docker-compose 3.8
Dev tooling	nodemon (hot-reload in dev)
Package manager	npm (CommonJS modules, package-lock.json)

1.4 High-Level Architecture

The backend follows a **modular monolith** architecture. Each domain (series, team, player, match, score, commentary ...) is self-contained in its own folder under `src/modules/`. Modules expose a consistent four-layer stack:

Route	Declares HTTP verb + path, wires middleware (auth, validate) to controller handler
Controller	Thin adapter - calls service, serialises response to JSON, delegates errors via <code>asyncHandler</code>
Service	All business logic, cross-module coordination, error throwing, socket emission
Repository	Pure DB calls (Mongoose) - zero business logic, zero HTTP knowledge

A **public user/ sub-tree** inside modules mirrors the admin modules but is read-only, unauthenticated, and caches responses. The two trees share the same Mongoose models - there is no data duplication.

Communication between modules happens through direct **`require()` imports** of repository/service files - no IoC container at runtime, though a static `container.js` registers all singletons for documentation purposes.

2. Complete Folder Structure

```

server/
  Dockerfile
  docker-compose.yml
  .dockerignore
  .gitignore
  package.json
  package-lock.json
  debug-output.txt
  seed/
  ■■■ index.js
  ■■■ src/
    ■■■ server.js          ← Entry point (HTTP + Socket.IO + DB)
    ■■■ app.js             ← Express app, route wiring, middleware
    ■■■ config/
      ■■■ db.js            ← MongoDB connection
      ■■■ container.js     ← Service/repository registry
    ■■■ sockets/
      ■■■ socketGateway.js ← Socket.IO init + emitToMatch helper
    ■■■ shared/
      ■■■ constants/
        ■■■ roles.js
        ■■■ matchStatus.js
      ■■■ errors/
        ■■■ AppError.js, BadRequestError.js, ConflictError.js
        ■■■ ForbiddenError.js, NotFoundError.js, UnauthorizedError.js
        ■■■ index.js
      ■■■ middleware/
        ■■■ auth.js        ← authenticate / authorize
        ■■■ errorHandler.js
        ■■■ notFound.js
        ■■■ validateRequest.js
      ■■■ utils/
        ■■■ asyncHandler.js
      ■■■ validators/
        ■■■ common.js      ← objectId, idParamSchema
    ■■■ modules/
      ■■■ auth/            ← register / login
      ■■■ users/           ← user CRUD (SUPER_ADMIN only)
      ■■■ series/          ← series CRUD (admin)
      ■■■ team/            ← team CRUD (admin)
      ■■■ squad/           ← squad management (admin)
      ■■■ player/          ← player CRUD (admin)
      ■■■ match/           ← match lifecycle (admin+scorer)
      ■■■ playing-xi/       ← XI selection (admin+scorer)
      ■■■ score/           ← live scoring (admin+scorer)
      ■■■ commentary/      ← ball commentary (admin+scorer)
      ■■■ user/            ← PUBLIC read-only API
        ■■■ cache/
          ■■■ responseCache.js
        ■■■ shared/
          ■■■ query.js      ← ensureId, pagination, escapeRegex
          ■■■ respond.js    ← controller factory
        ■■■ home/          ← GET /api/home
        ■■■ match/         ← GET /api/matches
        ■■■ series/        ← GET /api/series
        ■■■ team/          ← GET /api/teams
        ■■■ player/        ← GET /api/players
        ■■■ commentary/    ← GET /api/matches/:id/commentary
        ■■■ search/        ← GET /api/search
        ■■■ points-table/  ← GET /api/series/:id/points-table
  
```

Each admin module contains:

```

.model.js      Mongoose schema + model
  
```

```
.repository.js    DB access layer
.service.js       Business logic
.controller.js    HTTP adapter
.route.js         Express router
dto/.dto.js       Data Transfer Object (re-exports validators)
interfaces/.interface.js  Field lists / type metadata
validators/.validator.js  Zod schemas
```

3. Folder-by-Folder Analysis

3.1 server/ (root)

The root of the deployable unit. Contains Docker configuration, npm manifest, and the seed script. Acts as the container for the entire Node.js application.

Dockerfile

Builds a production node:20-alpine image, installs npm deps, exposes port 3000, includes HTTP healthcheck.

docker-compose.yml

Defines two services: *app* (Express) and *mongodb* (Mongo 7). App waits for MongoDB healthcheck. Named volumes persist data across restarts.

.dockerignore

Excludes node_modules and .env from Docker context to keep image small.

package.json

Declares all 10 runtime deps, three npm scripts (dev/start/seed). No devDependencies (nodemon expected globally or in Docker layer).

3.2 src/config/

Application bootstrap configuration. Two files, both required before routes load.

db.js

Async function connectDB() uses mongoose.connect(). Falls back to localhost/cricbuzz_backend if MONGODB_URI env is absent. Called in server.js before listen().

container.js

Static service locator that eagerly requires every repository and service. Not used at runtime for injection - serves as a central manifest and documentation aid.

3.3 src/modules/auth/

Handles user registration and login. No protected routes - these are the entry points for all credential flows.

auth.service.js

signToken() builds a 7-day JWT (id, role, email). register() checks email uniqueness, bcrypt-hashes password, creates user. login() compares hash, returns token.

auth.validator.js

Zod: registerSchema requires name, email, password (min 6), optional role enum. loginSchema requires email + password.

auth.route.js

POST /api/auth/register and POST /api/auth/login. Both pass through validateRequest middleware before controller.

3.4 src/modules/users/

User management accessible exclusively to SUPER_ADMIN. Provides read-only access - no update/delete exposed (soft-delete only via model flag).

user.model.js

Fields: name, email (unique, lowercase), password (hashed), role (enum), isDeleted (soft-delete flag). Mongoose timestamps add createdAt/updatedAt.

user.repository.js

create(), findByEmail() (active only), findById() (excludes password), findAll() (excludes password).

user.service.js

Thin delegation to repository. getUsers() and getUserById().

3.5 src/modules/series/

Full CRUD for cricket series. Enforces uniqueness on both name and season fields. Prevents deletion when child matches exist.

series.model.js

Fields: name (unique), shortName, season (unique), status enum (UPCOMING/LIVE/COMPLETED), logo URL, audit refs (createdBy/updatedAt).

series.service.js

createSeries() checks both name and season uniqueness. updateSeries() allows partial update with same uniqueness check. deleteSeries() guards against orphan matches via matchRepository.exists().

3.6 src/modules/team/

Team entity management. Embeds a squadPlayers array of Player ObjectId refs directly on the Team document.

team.model.js

Fields: name (unique), shortName (unique), logo (required), primaryColor, squadPlayers: [ObjectId→Player], audit refs, isDeleted.

team.repository.js

addPlayer() uses \$addToSet (idempotent). removePlayer() uses \$pull. Both are atomic MongoDB operations.

3.7 src/modules/player/

Global player catalogue. Players are created independently then assigned to team squads.

player.model.js

Fields: name, image URL, role enum (BATSMAN/BOWLER/ALL_ROUNDERS/WICKET_KEEPER), country, battingStyle, bowlingStyle (optional for pure batsmen), isDeleted, audit refs.

3.8 src/modules/squad/

Nested resource under teams. Manages which players belong to a team's registered squad. Squad size gates match creation (min 11 required).

squad.service.js

`addPlayerToSquad()` verifies both team and player exist, then checks for duplicates before calling `teamRepository.addPlayer()`. `removePlayerFromSquad()` requires team existence check only.

squad.route.js

Mounted at `/api/teams/:teamId/squad` with `mergeParams: true` so `teamId` is available inside router.

3.9 src/modules/match/

Central domain - orchestrates the full match lifecycle. Most complex service in the codebase.

match.model.js

Embeds `playingXI` as nested `{ player, isCaptain, isWicketKeeper }` objects (no separate collection). Status enum enforces lifecycle ordering. Four compound MongoDB indexes optimise common query patterns.

match.service.js

`createMatch()` validates squads have ≥ 11 players. `updateToss()` transitions `UPCOMING` $\rightarrow$ `TOSS_COMPLETED`. `startMatch()` requires `PLAYING_XI_SELECTED`. `completeMatch()` requires `LIVE` or `INNINGS_BREAK`. Every state change emits a `Socket.IO` event.

3.10 src/modules/playing-xi/

Validates and persists the 11-player selection for both teams in a match. Transitions match status to `PLAYING_XI_SELECTED`.

playingXi.service.js

`validateTeamXi()` enforces: exactly 11 players, no duplicates, all in squad, exactly 1 captain, exactly 1 wicket-keeper. `selectPlayingXi()` verifies match is in `TOSS_COMPLETED` state then persists and emits.

3.11 src/modules/score/

Innings-level score tracking. Score documents are created per innings during `LIVE` matches. Emits `score.updated` via `Socket.IO` on every write.

score.model.js

Fields: `matchId` (ref), `innings` (1 or 2), `battingTeam` (ref), `score`, `wickets` (0-10), `overs` (regex validated string format `X.Y`), `runRate`, `target` (optional for 2nd innings).

score.service.js

`ensureLiveMatch()` guard is called on both create and update. Emits to socket room after every successful write.

3.12 src/modules/commentary/

Ball-by-ball commentary. Supports five event types. Paginated for public consumption.

commentary.model.js

Fields: matchId, over (number), ball (1-6), text, type enum (NORMAL/FOUR/SIX/WICKET/MILESTONE). Compound index on {matchId, createdAt:-1} ensures efficient reverse-chronological queries.

commentary.service.js

addCommentary() enforces LIVE match, creates doc, emits commentary.created. deleteCommentary() is admin-only (fix errors).

3.13 src/modules/user/ (Public API)

Read-only, unauthenticated public-facing API. Mirrors the admin domain but with richer population queries, filtering, and caching. No writes, no auth middleware.

home/

Returns three categorised match lists (live/upcoming/recent) in a single request. Optimised with Promise.all.

match/

getMatches() supports ?status=live|upcoming|completed filter. getMatch() returns match + scores. getCenter() bundles matchInfo + liveScore + playingXI + result for the live-match centre UI. getScorecard() returns innings-split scores.

series/

getSeries() lists all. getSeriesDetails() returns series + matches + points table in one response.

commentary/

Paginated feed (default 50/page). Validates matchId is a valid ObjectId before querying.

points-table/

Derives standings on-the-fly from completed matches. No separate collection.

search/

Parallel query across Player, Team, Series with regex. Min 2 chars, max 10 results each.

cache/responseCache.js

In-process Map keyed by req.originalUrl. TTL configurable per router mount. Patches res.json() to intercept successful responses.

shared/query.js

ensureId() validates MongoDB ObjectId format (throws 404 on invalid). escapeRegex() sanitises search input. pagination() parses page/limit with safe defaults.

3.14 src/shared/

Cross-cutting infrastructure used by every module.

constants/

roles.js: SUPER_ADMIN, ADMIN, SCORER. matchStatus.js: full 7-state lifecycle enum.

errors/

AppError base class with statusCode + isOperational flag. Five subclasses: BadRequest(400), Unauthorized(401), Forbidden(403), NotFound(404), Conflict(409). Barrel index.js re-exports all.

middleware/

auth.js: authenticate() extracts Bearer token, verifies JWT, attaches req.user. authorize(...roles) checks req.user.role inclusion. errorHandler.js: reads err.statusCode or defaults to 500. validateRequest(schema): runs Zod safeParse, attaches result to req.validated. notFound.js: 404 fallback.

utils/asyncHandler.js

Wraps async route handlers in Promise.resolve().catch(next) - eliminates try/catch boilerplate.

validators/common.js

Exports Zod instance, objectId regex validator, idParamSchema (params.id must be 24-char hex).

3.15 src/sockets/

Socket.IO gateway. Single module that owns the io instance and provides a clean emitToMatch() helper usable from any service.

socketGateway.js

initSocket(io) registers join-match / leave-match events (two aliases each for compatibility). emitToMatch(matchId, event, payload) emits to the match:{matchId} room. ioInstance held in module scope.

3.16 seed/

Development/testing database seeder. Destructively clears all collections then inserts realistic BGT-2026 data.

index.js

Inserts SUPER_ADMIN + SCORER users, Border-Gavaskar Trophy 2026 series, 11 Indian + 11 Australian players, two teams with squad assignments, a LIVE match with full playingXI, a score record, and two commentary entries.

4. File-by-File Analysis (Key Files)

src/server.js - Application Entry Point

This is the first file Node.js executes (`node ./src/server.js`). It bootstraps the entire process.

Imports	http (native), socket.io Server, express app, connectDB, initSocket
Creates	http.Server wrapping the Express app, then passes it to new Server() for Socket.IO
Startup sequence	1. connectDB() → 2. server.listen(PORT) - server only starts after DB is ready
Error handling	DB connection failure calls process.exit(1) - container restart policy handles recovery
Socket.IO CORS	Reads CORS_ORIGIN env; defaults to * (should be restricted in production)

src/app.js - Express Application Factory

Creates and configures the Express application. All route mounting happens here. Does NOT start the HTTP server (that is server.js's job).

Global middleware	cors() (no options = allow all origins), express.json() with 5 MB body limit
Health endpoint	GET /health - returns 200 JSON, used by Docker healthcheck and load balancers
Route ordering	Public user routes are mounted BEFORE admin routes on the same path prefix. Express's first-match-wins ensures public GETs are served by the cached, unauthenticated handlers. Admin POST/PATCH/DELETE fall through to the second mount.
Error middleware	notFound (404) and errorHandler (global) are registered last in the chain
Cache middleware	responseCache(ttl) is applied per-router at mount time, not globally

src/shared/middleware/auth.js

authenticate()	Reads Authorization: Bearer header. jwt.verify() with JWT_SECRET env. Attaches decoded payload to req.user. Throws UnauthorizedError on missing or invalid token.
authorize(...roles)	Higher-order function returning middleware. Checks req.user.role is in the allowed roles array. Throws ForbiddenError if not.
Security note	JWT_SECRET falls back to 'change-this-secret' - MUST be overridden in production via environment variable.

src/shared/middleware/validateRequest.js

Implements schema-first request validation using Zod. Every route that writes data passes through this middleware.

Input	A Zod schema that describes { body, params, query }
--------------	---

Process	Calls <code>schema.safeParse({ body, params, query })</code> - does not throw on failure
On failure	Returns 400 JSON with <code>{ success:false, message:'Validation failed', errors: result.error.flatten() }</code>
On success	Attaches validated data to <code>req.validated</code> - controllers read <code>req.validated.body</code> / <code>req.validated.params</code>

src/sockets/socketGateway.js

Pattern	Module-scope singleton - <code>ioInstance</code> set once by <code>initSocket()</code> , then used by <code>emitToMatch()</code> across all service calls
Rooms	Each match gets a room named <code>match:{matchId}</code> . Clients join/leave with <code>'join-match'</code> / <code>'leave-match'</code> events.
Events emitted	<code>toss.updated</code> , <code>playingXI.updated</code> , <code>match.started</code> , <code>match.completed</code> , <code>score.updated</code> , <code>commentary.created</code>
Null guard	<code>emitToMatch()</code> is a no-op if <code>ioInstance</code> is null (e.g., during unit tests) or <code>matchId</code> is falsy

src/modules/user/cache/responseCache.js

Type	In-process (non-distributed) Map-based TTL cache
Key	<code>req.originalUrl</code> - full URL including query string
Stores	<code>{ status, body, expiresAt }</code> per URL
Bypass	Non-GET requests skip cache entirely
Invalidation	Pure TTL - no manual invalidation on writes. Score/commentary updates via <code>Socket.IO</code> compensate.
Limitation	Cache is per-process. Horizontal scaling (multiple instances) breaks cache consistency - requires Redis migration.
TTL values	10s: home, matches 5s: commentary 30s: search, points-table 60s: series, teams, players

src/modules/user/points-table/pointsTable.service.js

Derives the points table entirely from match results - no dedicated collection. Algorithm:

- Fetch all COMPLETED matches for the series (populated with team/winner refs)
- Use a Map keyed by team ObjectId string to accumulate stats
- For each match: increment played for both teams; increment won (+2 points) for winner, lost for loser
- Sort by points desc, then won desc

This approach is correct and efficient for typical tournament sizes (< a few hundred matches). For very large leagues, a materialised view would be preferable.

5. Application Flow

5.1 Admin Write Flow (e.g. Update Score)

Step	Component	Detail
1	Client (scorer UI)	PATCH /api/scores/:id with Authorization: Bearer <jwt> and JSON body
2	Express Router (score.route.js)	authenticate middleware verifies JWT → authorize(SUPER_ADMIN, SCORER) check
3	validateRequest(updateScoreSchema)	Zod validates body fields; attaches req.validated
4	score.controller.js	asyncHandler wraps: calls scoreService.updateScore(id, req.validated.body, req.user)
5	score.service.js	ensureLiveMatch() → scoreRepository.updateById() → emitToMatch(matchId, 'score.
6	score.repository.js	Match.findOneAndUpdate() with { new: true } - returns updated document
7	socketGateway.js	ioInstance.to('match:{id}').emit('score.updated', payload) - all subscribed clients receive
8	Controller response	res.json({ success: true, data: updatedScore })
9	Fan client (Socket.IO)	Receives score.updated event and updates UI without polling

5.2 Public Read Flow (e.g. GET /api/home)

1. Client request	GET /api/home (no auth header required)
2. responseCache(10)	Checks Map for req.originalUrl. On cache hit returns immediately (avg <1ms).
3. Cache miss	Calls next() - continues to home.controller.js
4. home.service.js	Calls home.repository.getHomeMatches() - three parallel Mongoose queries
5. MongoDB	Three queries: LIVE/INNINGS_BREAK matches, upcoming-state matches, COMPLETED matches (each limited to 10)
6. Populated response	Series, team1, team2, tossWinner, winner fields populated with name/shortName/logo
7. Cache store	res.json override stores body + expiresAt in Map, then sends response
8. Next 10 seconds	All subsequent GET /api/home requests served from Map without touching MongoDB

5.3 Match Lifecycle State Machine

Matches progress through a strict ordered sequence. Each transition is enforced in match.service.js with explicit status checks:

Status	Meaning	Transition
UPCOMING	Initial state when match is created	→ TOSS_COMPLETED via PATCH /:id/toss
TOSS_COMPLETED	Toss winner and decision recorded	→ PLAYING_XI_SELECTED via POST /matches/:id/playing-xi
PLAYING_XI_SELECTED	Both XIs validated and saved	→ LIVE via PATCH /:id/start

LIVE	Match in progress, scoring active	→ INNINGS_BREAK (manual) or COMPLETED via PATCH /:id/
INNINGS_BREAK	Between innings	→ LIVE (re-open) or COMPLETED
COMPLETED	Final state, winner recorded	Terminal - no further transitions
DRAFT	Defined in constants, not yet used in service logic	Reserved for future use

6. Architecture Analysis

6.1 Backend Architecture - Layered Modular Monolith

The codebase implements a clean four-layer architecture within each domain module. Dependency direction is strictly top-down: Route → Controller → Service → Repository. No layer imports from a layer above it.

Why a modular monolith over microservices? Cricket scoring is stateful and highly relational (match references series, teams, players, scores, commentary). A monolith avoids distributed transaction complexity while still being well-structured for a future split if scale demands it.

6.2 Authentication & Authorization Flow

Registration	POST /api/auth/register → validate → bcrypt.hash(password, 10) → User.create() → signToken() → return {token, user}
Login	POST /api/auth/login → validate → User.findByEmail() → bcrypt.compare() → signToken() → return {token, user}
Token structure	JWT payload: { id, role, email }. Signed with JWT_SECRET. Expires in JWT_EXPIRES_IN (default 7d).
Protected request	Client sends Authorization: Bearer → authenticate middleware → jwt.verify() → req.user = payload
Role check	authorize(ROLES.ADMIN, ROLES.SUPER_ADMIN) → checks req.user.role in allowed array
Three roles	SUPER_ADMIN: all operations. ADMIN: series/team/squad/player/match CRUD. SCORER: toss, playing-XI, score, commentary.

6.3 Database Architecture

ODM	Mongoose 9 - schema validation, middleware hooks, population, virtuals
Connection	Single connection per process managed by Mongoose's connection pool
Soft delete	All domain models have isDeleted:Boolean (default false). All repository queries append {isDeleted:false} filter.
Audit trail	createdBy and updatedBy (User ObjectId refs) on every domain model. Populated via service audit() helpers.
Indexes	Match model has 4 compound indexes covering: status+startTime, seriesId+startTime, team1+startTime, team2+startTime. Score: {matchId, innings}. Commentary: {matchId, createdAt:-1}.
Embedding vs reference	playingXI is embedded in Match (no separate collection) - correct choice as XI never changes after selection.

6.4 API Architecture

Style	REST - resource-oriented URLs, HTTP verbs as actions
Versioning	None currently - all routes under /api/
Response envelope	{ success: true/false, data: ... } or { success:false, message:'...', errors:{...} }
Error codes	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal

Body limit	5 MB (for base64-encoded images in player/team creation)
Public vs admin	Same path prefix, different mount order. Public handlers (GET, no auth) registered first; admin handlers fall through.

6.5 Real-Time Architecture

Transport	Socket.IO 4 over WebSocket (with HTTP long-poll fallback)
Room model	One room per match ID: 'match:{matchId}'. Fan clients join when viewing a match.
Event bus	emitToMatch() is called from service layer after successful DB writes - guarantees event only fires on success.
Events	toss.updated, playingXI.updated, match.started, match.completed, score.updated, commentary.created
Client events	join-match, leave-match (two aliases each: dot and dash notation for client compatibility)
Scalability note	Single-process io instance - Socket.IO adapter must be changed to Redis adapter for multi-instance deployments.

7. API Documentation

7.1 Auth Routes /api/auth

Method	POST
Route	/api/auth/register
Purpose	Register new admin/scorer user
Auth	None (public)
Body/Params	{ name, email, password, role? }
Response	{ token, user: { id, name, email, role } }

Method	POST
Route	/api/auth/login
Purpose	Login and receive JWT
Auth	None (public)
Body/Params	{ email, password }
Response	{ token, user: { id, name, email, role } }

7.2 User Routes /api/users

All endpoints require authenticate + authorize(SUPER_ADMIN)

Method	GET
Route	/api/users
Purpose	List all active users
Auth	SUPER_ADMIN
Body/Params	-
Response	Array of user objects (no passwords)

Method	GET
Route	/api/users/:id
Purpose	Get single user
Auth	SUPER_ADMIN
Body/Params	-
Response	User object

7.3 Series Routes /api/series

Admin routes require authenticate + authorize(SUPER_ADMIN | ADMIN)

Method	GET
--------	-----

Route	/api/series
Purpose	List all series
Auth	Admin
Body/Params	-
Response	Array of series
Method	GET
Route	/api/series/:id
Purpose	Get series by ID
Auth	Admin
Body/Params	-
Response	Series object
Method	POST
Route	/api/series
Purpose	Create series
Auth	SUPER_ADMIN ADMIN
Body/Params	{ name, shortName, season, status?, logo? }
Response	Created series
Method	PATCH
Route	/api/series/:id
Purpose	Update series
Auth	SUPER_ADMIN ADMIN
Body/Params	Partial series fields
Response	Updated series
Method	DELETE
Route	/api/series/:id
Purpose	Soft-delete series (guards if matches exist)
Auth	SUPER_ADMIN ADMIN
Body/Params	-
Response	Deleted series

7.4 Team Routes /api/teams

Method	GET
Route	/api/teams
Purpose	List all teams
Auth	Admin

Body/Params	-
Response	Array of teams
Method	GET
Route	/api/teams/:id
Purpose	Get team by ID
Auth	Admin
Body/Params	-
Response	Team object
Method	POST
Route	/api/teams
Purpose	Create team
Auth	SUPER_ADMIN ADMIN
Body/Params	{ name, shortName, logo, primaryColor? }
Response	Created team
Method	PATCH
Route	/api/teams/:id
Purpose	Update team
Auth	SUPER_ADMIN ADMIN
Body/Params	Partial team fields
Response	Updated team
Method	DELETE
Route	/api/teams/:id
Purpose	Soft-delete team
Auth	SUPER_ADMIN ADMIN
Body/Params	-
Response	Deleted team

7.5 Squad Routes /api/teams/:teamId/squad

Method	GET
Route	/api/teams/:teamId/squad
Purpose	Get team squad
Auth	SUPER_ADMIN ADMIN
Body/Params	-
Response	Array of player objects
Method	POST

Route	/api/teams/:teamId/squad
Purpose	Add player to squad
Auth	SUPER_ADMIN ADMIN
Body/Params	{ playerId }
Response	Updated team
Method	DELETE
Route	/api/teams/:teamId/squad/:playerId
Purpose	Remove player from squad
Auth	SUPER_ADMIN ADMIN
Body/Params	-
Response	Updated team

7.6 Player Routes /api/players

Method	GET
Route	/api/players
Purpose	List all players
Auth	Admin
Body/Params	-
Response	Array of players
Method	GET
Route	/api/players/:id
Purpose	Get player by ID
Auth	Admin
Body/Params	-
Response	Player object
Method	POST
Route	/api/players
Purpose	Create player
Auth	SUPER_ADMIN ADMIN
Body/Params	{ name, role, country, image?, battingStyle?, bowlingStyle? }
Response	Created player
Method	PATCH
Route	/api/players/:id
Purpose	Update player
Auth	SUPER_ADMIN ADMIN

Body/Params	Partial player fields
Response	Updated player
Method	DELETE
Route	/api/players/:id
Purpose	Soft-delete player
Auth	SUPER_ADMIN ADMIN
Body/Params	-
Response	Deleted player

7.7 Match Routes /api/matches

Method	GET
Route	/api/matches
Purpose	List all matches
Auth	Any authenticated
Body/Params	-
Response	Array of matches
Method	GET
Route	/api/matches/:id
Purpose	Get match by ID
Auth	Any authenticated
Body/Params	-
Response	Match object
Method	POST
Route	/api/matches
Purpose	Create match
Auth	SUPER_ADMIN ADMIN
Body/Params	{ seriesId, team1, team2, venue, startTime, matchNumber? }
Response	Created match
Method	PATCH
Route	/api/matches/:id
Purpose	Update match details
Auth	SUPER_ADMIN ADMIN
Body/Params	Partial match fields
Response	Updated match
Method	DELETE

Route	/api/matches/:id
Purpose	Soft-delete match
Auth	SUPER_ADMIN ADMIN
Body/Params	-
Response	Deleted match

Method	PATCH
Route	/api/matches/:id/toss
Purpose	Record toss result - UPCOMING→TOSS_COMPLETED
Auth	SUPER_ADMIN SCORER
Body/Params	{ tossWinner, tossDecision }
Response	Updated match + socket emit

Method	PATCH
Route	/api/matches/:id/start
Purpose	Start match - PLAYING_XI_SELECTED→LIVE
Auth	SUPER_ADMIN SCORER
Body/Params	-
Response	Updated match + socket emit

Method	PATCH
Route	/api/matches/:id/complete
Purpose	Complete match - LIVE/INNINGS_BREAK→COMPLETED
Auth	SUPER_ADMIN SCORER
Body/Params	{ winner, result }
Response	Updated match + socket emit

7.8 Playing XI /api/matches/:id/playing-xi

Method	POST
Route	/api/matches/:id/playing-xi
Purpose	Select playing XI for both teams. Validates 11 players, 1 captain, 1 WK, all in squad.
Auth	SUPER_ADMIN SCORER
Body/Params	{ team1: [{player, isCaptain, isWicketKeeper}×11], team2: same }
Response	Updated match + socket emit

7.9 Score Routes /api/scores

Method	POST
--------	------

Route	/api/scores
Purpose	Create innings score (match must be LIVE)
Auth	SUPER_ADMIN SCORER
Body/Params	{ matchId, innings, battingTeam, score, wickets, overs, runRate, target? }
Response	Created score + socket emit
Method	PATCH
Route	/api/scores/:id
Purpose	Update innings score (match must be LIVE)
Auth	SUPER_ADMIN SCORER
Body/Params	Partial score fields
Response	Updated score + socket emit
Method	GET
Route	/api/scores/match/:matchId
Purpose	Get all scores for a match
Auth	SUPER_ADMIN SCORER
Body/Params	-
Response	Array of score objects

7.10 Commentary Routes /api/commentary

Method	POST
Route	/api/commentary
Purpose	Add ball commentary (match must be LIVE)
Auth	SUPER_ADMIN SCORER
Body/Params	{ matchId, over, ball, text, type? }
Response	Created commentary + socket emit
Method	DELETE
Route	/api/commentary/:id
Purpose	Delete commentary entry
Auth	SUPER_ADMIN SCORER
Body/Params	-
Response	Deleted entry
Method	GET
Route	/api/commentary/match/:matchId
Purpose	List commentary for a match
Auth	SUPER_ADMIN SCORER

Body/Params	-
Response	Array of commentary

7.11 Public Routes (No Auth)

Method	Route	Description	Cache
GET	/api/home	Home feed: { liveMatches, upcomingMatches, recentMatches }	10s TTL
GET	/api/matches	List matches. Query: ?status=live upcoming completed	10s TTL
GET	/api/matches/:matchId	Match details + scores	10s TTL
GET	/api/matches/:matchId/center	Live centre: matchInfo + liveScore + playingXI + result	10s TTL
GET	/api/matches/:matchId/scorecard	Innings breakdown: { innings1, innings2 }	10s TTL
GET	/api/matches/:matchId/commentary	Paginated commentary. Query: ?page=1&limit=50	5s TTL
GET	/api/series	List all series	60s TTL
GET	/api/series/:seriesId	Series details + matches + points table	60s TTL
GET	/api/series/:seriesId/matches	Matches for a series	60s TTL
GET	/api/series/:seriesId/points-table	Derived points table	30s TTL
GET	/api/teams	List all teams	60s TTL
GET	/api/teams/:teamId	Team details	60s TTL
GET	/api/players	List all players	60s TTL
GET	/api/players/:playerId	Player details	60s TTL
GET	/api/search?q=...	Search players, teams, series (min 2 chars)	30s TTL

8. Database Documentation

8.1 User Model (collection: users)

Field	Type	Required	Default/Constraint	Purpose
name	String	Yes	trim:true	Display name
email	String	Yes	unique, lowercase	Login identifier
password	String	Yes	-	bcrypt hash (cost 10)
role	String	No	SCORER	SUPER_ADMIN ADMIN SCORER
isDeleted	Boolean	No	false	Soft-delete flag
createdAt	Date	auto	-	Mongoose timestamps
updatedAt	Date	auto	-	Mongoose timestamps

8.2 Series Model (collection: series)

Field	Type	Required	Default/Constraint	Purpose
name	String	Yes	unique, trim	Full series name e.g. BGT 2026
shortName	String	Yes	trim	Abbreviation e.g. BGT
season	String	Yes	unique, trim	Year or season label
status	String	No	UPCOMING	UPCOMING LIVE COMPLETED
logo	String	No	-	Logo image URL
isDeleted	Boolean	No	false	Soft-delete
createdBy	ObjectId	No	-	Ref: User
updatedAt	ObjectId	No	-	Ref: User

8.3 Team Model (collection: teams)

Field	Type	Required	Default/Constraint	Purpose
name	String	Yes	unique, trim	Team full name
shortName	String	Yes	unique, trim	3-letter code e.g. IND
logo	String	Yes	-	Logo image URL
primaryColor	String	No	-	Hex colour code
squadPlayers	[ObjectId]	No	[]	Ref: Player - registered squad
isDeleted	Boolean	No	false	Soft-delete
createdBy/updatedBy	ObjectId	No	-	Ref: User

8.4 Player Model (collection: players)

Field	Type	Required	Default/Constraint	Purpose
name	String	Yes	trim	Player full name

image	String	No	-	Profile image URL
role	String	Yes	enum	BATSMAN BOWLER ALL_ROUNDER WICKET_KEEPER
country	String	Yes	trim	Nationality
battingStyle	String	No	-	e.g. 'Right-hand bat'
bowlingStyle	String	No	-	e.g. 'Right-arm fast'
isDeleted	Boolean	No	false	Soft-delete
createdBy/updatedBy	ObjectId	No	-	Ref: User

8.5 Match Model (collection: matches)

Field	Type	Required	Default/Constraint	Purpose
seriesId	ObjectId	Yes	Ref:Series	Parent series
matchNumber	String	No	-	e.g. '1st Test'
venue	String	Yes	trim	Stadium name + city
startTime	Date	Yes	-	Scheduled start
status	String	No	UPCOMING	7-state lifecycle enum
team1/team2	ObjectId	Yes	Ref:Team	Competing teams
tossWinner	ObjectId	No	-	Ref:Team - set after toss
tossDecision	String	No	BAT BOWL	Toss winner's choice
playingXI.team1/2	[embedded]	No	[]	{ player, isCaptain, isWicketKeeper }
winner	ObjectId	No	-	Ref:Team - set on completion
result	String	No	-	Result description text
isDeleted	Boolean	No	false	Soft-delete

Indexes: {status,startTime,isDeleted} · {seriesId,startTime,isDeleted} · {team1,startTime,isDeleted} · {team2,startTime,isDeleted}

8.6 Score Model (collection: scores)

Field	Type	Required	Default/Constraint	Purpose
matchId	ObjectId	Yes	Ref:Match	Parent match
innings	Number	Yes	min:1	1 or 2
battingTeam	ObjectId	Yes	Ref:Team	Team batting this innings
score	Number	No	0	Total runs
wickets	Number	No	0	Wickets fallen (0-10)
overs	String	No	0.0	Format validated X.Y (Y: 0-5)
runRate	Number	No	0	Current run rate
target	Number	No	-	2nd innings target (optional)

Index: {matchId, innings}

8.7 Commentary Model (collection: commentaries)

Field	Type	Required	Default/Constraint	Purpose
matchId	ObjectId	Yes	Ref:Match	Parent match
over	Number	Yes	min:0	Over number
ball	Number	Yes	min:1 max:6	Ball in over
text	String	Yes	trim	Commentary text
type	String	No	NORMAL	NORMAL FOUR SIX WICKET MIL ESTONE

Index: {matchId, createdAt:-1} - optimises reverse-chronological feed

8.8 Entity Relationship Summary

Entity	References	Relationship
User	-	Standalone; referenced as audit createdBy/updatedBy on all models
Series	User (audit)	One-to-many with Match
Team	Player (squadPlayers), User (audit)	Many-to-many with Series via Match; owns squad
Player	User (audit)	Many-to-many with Team via squadPlayers; embedded in Match
Match	Series, Team×2, Team (tossWinner/winner), User (audit)	One-to-many with Score and Commentary
Score	Match, Team (battingTeam), User (audit)	One Score per innings per match (1 or 2)
Commentary	Match, User (audit)	Many entries per match, paginated in reverse order

9. Configuration Files

9.1 package.json

name	express-backend
type	commonjs - all files use require/module.exports (no ESM)
main	index.js (not used directly - entry is src/server.js via scripts)
scripts	dev: nodemon ./src/server.js start: node ./src/server.js seed: node ./seed/index.js
Runtime deps	express, mongoose, jsonwebtoken, bcryptjs, zod, socket.io, dotenv, cors, axios, cheerio, puppeteer
Notable	No devDependencies declared - nodemon expected globally or handled by Docker layer

9.2 Dockerfile

Base	node:20-alpine - minimal Linux image, significantly smaller than node:20
Build steps	1. COPY package.json → 2. npm install → 3. COPY . . → 4. EXPOSE 3000 → 5. CMD npm run dev
Healthcheck	HTTP GET localhost:3000 every 30s, timeout 10s, 3 retries - starts after 5s
Note	CMD runs dev (nodemon) - production should override to npm start to remove nodemon dependency

9.3 docker-compose.yml

app service	Builds from Dockerfile. MONGODB_URI points to internal mongodb service. Waits for mongodb healthcheck before starting. Volume mount ./src:/app/src enables hot-reload.
mongodb service	Official mongo:7.0 image. Healthcheck uses mongosh ping. Named volumes persist data and config across container restarts.
Network	Bridge network app-network isolates services from host.
Restart	unless-stopped - containers restart on crash but not on manual docker compose down.

9.4 Environment Variables

Variable	Default	Notes
PORT	3000	HTTP server port
MONGODB_URI	mongodb://localhost:27017/cricbuzz_backend	MongoDB connection string
JWT_SECRET	change-this-secret	■ MUST be overridden in production
JWT_EXPIRES_IN	7d	Token expiry duration
CORS_ORIGIN	*	■ MUST be restricted to frontend domain in production

NODE_ENV	development	Set to production in docker-compose
----------	-------------	-------------------------------------

10. Security Analysis

10.1 Authentication Security

Password hashing	bcryptjs with cost factor 10 (~100ms per hash). Timing-safe comparison via bcrypt.compare().
Token signing	HS256 (HMAC-SHA256). Payload: { id, role, email }. Default secret is INSECURE - must change.
Token lifetime	7 days by default. No refresh token mechanism - compromised tokens remain valid until expiry.
Token storage	Not managed by server - client responsibility. Headers only (Bearer), not cookies.

10.2 Authorization Security

RBAC enforcement	authorize() middleware checks role on every protected route - no capability escapes to controller layer.
Public routes	Intentionally unauthenticated. No write operations exposed publicly.
Scorer scope	SCORER can only perform live-match operations (toss, XI, score, commentary) - cannot modify master data.

10.3 Input Validation

Library	Zod 4 - schema-first, safe-parse (no exceptions on invalid input).
Coverage	All POST/PATCH bodies validated. Route params validated (objectId regex). Query params validated where used.
ObjectId validation	custom regex /^[0-9a-fA-F]{24}\$/ - prevents MongoDB cast errors and injection attempts.
Regex search input	escapeRegex() in search module sanitises user input before building RegExp - prevents ReDoS.

10.4 Identified Security Risks

Severity	Risk	Description	Recommendation
CRITICAL	JWT_SECRET default value	'change-this-secret' is publicly known. Any attacker can forge valid tokens.	Set a long random secret in production .env (e.g. openssl rand -hex 32).
CRITICAL	CORS: *	All origins accepted. Any website can make authenticated requests.	Set CORS_ORIGIN to the specific frontend domain(s).
HIGH	No refresh token	7-day tokens cannot be revoked on logout/compromise.	Implement token blacklist (Redis) or short-lived tokens + refresh flow.

HIGH	In-memory cache	Cache is per-process and lost on restart. Multi-instance deployments share stale data.	Migrate to Redis with appropriate TTLs.
MEDIUM	Docker CMD uses nodemon	nodemon is a dev tool - not appropriate for production containers.	Override CMD to 'npm start' in production.
MEDIUM	No rate limiting	Auth endpoints (login/register) are not rate-limited - vulnerable to brute force.	Add express-rate-limit on auth routes (e.g. 10 req/min per IP).
LOW	Body limit 5MB	Large payloads accepted - could be abused for DoS.	5MB is generous for image URLs; consider reducing or using pre-signed upload URLs.
LOW	No helmet.js	HTTP security headers (X-Content-Type-Options etc.) not set.	Add helmet() middleware to app.js.

11. Real-Time Architecture (Socket.IO)

The real-time layer uses Socket.IO 4 in a room-based broadcast pattern. Every match has its own isolated room - fans subscribed to different matches never receive each other's events.

11.1 Connection Flow

Client connects	socket.connect() → Socket.IO handshake (WebSocket upgrade or long-poll fallback)
Join match	Client emits 'join-match' with matchId → server calls socket.join('match:{matchId}')
Leave match	Client emits 'leave-match' with matchId → server calls socket.leave('match:{matchId}')
Score update	Admin scorer updates score → service calls emitToMatch(matchId, 'score.updated', data)
Fan receives	All sockets in 'match:{matchId}' room receive 'score.updated' event with new score payload

11.2 Event Catalogue

Event	Trigger	Payload
toss.updated	Admin	Toss recorded - match status, tossWinner, tossDecision
playingXI.updated	Admin	Playing XI selected - full playingXI embedded in match
match.started	Admin	Match is now LIVE - status change payload
match.completed	Admin	Match finished - winner, result, status
score.updated	Scorer	Innings score changed - full Score document
commentary.created	Scorer	New ball commentary - Commentary document

12. Caching Architecture

The responseCache middleware provides an in-memory HTTP-layer cache for all public GET routes. It operates by intercepting `res.json()` to capture successful responses before they are sent, and returning cached copies on subsequent requests within the TTL window.

12.1 Cache Configuration by Route

Route Prefix	TTL	Reason
/api/home	10s	Live match status changes frequently
/api/matches (incl. center, scorecard)	10s	Live score data
/api/matches/:id/commentary	5s	Ball-by-ball updates, shortest TTL
/api/series/:id/points-table	30s	Changes only when match completes
/api/search	30s	Mostly stable data
/api/series	60s	Series rarely change
/api/teams	60s	Teams rarely change
/api/players	60s	Players rarely change

Design trade-off: Cache staleness is acceptable because real-time Socket.IO events push live updates to subscribed clients. A fan watching a live match gets immediate score/commentary pushes via WebSocket regardless of the 5-10 second cache window on the REST endpoints.

13. Complete Execution Flow (Server Startup)

Step	Location	Action
1	node src/server.js	Node.js loads server.js as the entry point. Process starts.
2	require('./app')	app.js loads - dotenv.config() reads .env file into process.env.
3	Express app created	cors() and express.json() middleware attached globally.
4	All routes required	Every route file is require()'d - Mongoose models are loaded, validators compiled, services i
5	GET /health registered	Simple health probe endpoint attached before all domain routes.
6	Public routes mounted	User-facing GET routes mounted with responseCache middleware.
7	Admin routes mounted	Auth-protected admin routes mounted (fall-through for same paths as public).
8	notFound + errorHandler	404 and global error middleware registered last.
9	http.Server created	Native http.createServer(app) wraps the Express app.
10	Socket.IO attached	new Server(httpServer) creates Socket.IO instance. initSocket(io) registers connection/join/l
11	connectDB()	mongoose.connect(MONGODB_URI) - async. If this rejects, process.exit(1).
12	server.listen(PORT)	Only called after DB connection succeeds. Console logs 'Server running on port 3000'.
13	Ready	Server accepts HTTP and WebSocket connections.

14. Developer Onboarding Guide

14.1 Prerequisites

- Node.js 20 LTS (<https://nodejs.org>)
- MongoDB 7 local instance OR Docker Desktop
- npm (bundled with Node.js)
- Git
- A REST client (Postman, Insomnia, or cURL)

14.2 Local Setup (Without Docker)

```
# 1. Clone repository
git clone && cd server

# 2. Install dependencies
npm install

# 3. Create environment file
cp .env.example .env # or create manually:
echo 'MONGODB_URI=mongodb://localhost:27017/cricbuzz_backend' >> .env
echo 'JWT_SECRET=your-dev-secret-here' >> .env
echo 'PORT=3000' >> .env

# 4. Seed database with test data
npm run seed

# 5. Start development server (hot-reload)
npm run dev

# Server available at http://localhost:3000
# Health check: GET http://localhost:3000/health
```

14.3 Docker Setup

```
# Start all services (Express + MongoDB)
docker compose up --build -d

# View logs
docker compose logs -f app

# Seed database inside container
docker compose exec app npm run seed

# Stop all services
docker compose down

# Stop and remove volumes (DELETES ALL DATA)
docker compose down -v
```

14.4 First API Calls

1. Register admin

```
POST /api/auth/register { name:'Admin', email:'a@test.com',
password:'pass123', role:'SUPER_ADMIN' }
```

2. Login	POST /api/auth/login { email:'a@test.com', password:'pass123' } → copy token from response
3. Authenticated call	GET /api/series with Authorization: Bearer
4. Public feed	GET /api/home (no auth needed)
5. Seeded admin creds	admin@cricket.com / password123 (after npm run seed)

14.5 Adding a New Module (Pattern Reference)

- **Step 1:** Create src/modules//.model.js - Mongoose schema with isDeleted, createdBy, updatedBy
- **Step 2:** Create .repository.js - pure DB functions (create, findAll, findById, updateById, softDelete)
- **Step 3:** Create .service.js - business logic, error throwing, cross-module repository calls
- **Step 4:** Create .controller.js - asyncHandler wrappers calling service, res.json response
- **Step 5:** Create validators/.validator.js - Zod schemas for create/update/param/query
- **Step 6:** Create .route.js - Express.Router with auth/validate middleware wired up
- **Step 7:** Register route in app.js with appropriate path prefix
- **Step 8:** Register in config/container.js for documentation purposes

14.6 Common Debugging Points

JWT errors

Check Authorization header is 'Bearer ' with a space. Verify JWT_SECRET matches between registration and current run.

Cast errors

MongoDB ObjectId cast error usually means an invalid ID string reached the DB. Wrap route params with validateRequest(idParamSchema) and use ensureId() in public services.

Match status errors

Follow the state machine strictly. A SCORER trying to add a score before match.status is LIVE will get 400.

Squad validation

createMatch() requires BOTH team squads to have ≥ 11 players. Seed or manually add players first.

Socket events not received

Ensure client calls socket.emit('join-match', matchId) after connecting. Check matchId matches the match being updated.

Cache stale data

Restart the server process to clear in-memory cache. Or lower TTL temporarily during development.

15. Project Summary & Recommendations

15.1 Strengths

- Clean modular architecture - each domain is independently understandable and testable
- Strict state-machine for match lifecycle - prevents data corruption from out-of-order operations
- Thorough input validation via Zod on every write endpoint - body, params, and query
- Consistent error hierarchy (AppError subclasses) with correct HTTP status codes
- Real-time Socket.IO integration that correctly fires events only after successful DB writes
- Soft-delete pattern across all models preserves audit history and prevents accidental data loss
- Smart route ordering in app.js allows public and admin routes to share URL prefixes cleanly
- In-memory cache significantly reduces MongoDB load for high-traffic public endpoints
- Production-ready Docker setup with healthchecks and persistent named volumes
- Comprehensive database seeder enables instant development environment setup

15.2 Weaknesses & Technical Debt

No test suite

Zero unit or integration tests. Adding any business logic change carries risk of regression.

In-memory cache

Not horizontally scalable. A second app instance has a cold cache and stale data.

No refresh tokens

7-day JWT with no revocation = security risk on credential compromise.

Insecure defaults

JWT_SECRET and CORS_ORIGIN must be overridden - no startup validation enforces this.

No rate limiting

Auth endpoints are brute-forceable without any throttling.

No pagination on admin list routes

GET /api/series, /api/teams, /api/players, /api/matches return ALL records - will degrade at scale.

No TypeScript

No compile-time safety. Interface files (*.interface.js) are documentation-only arrays, not enforced types.

puppeteer/cheerio dependency

Scraping utilities are bundled in production image (large footprint). Should be in a separate utility service.

No logging

console.log/console.error only. No structured logging (Winston/Pino), no request ID tracing.

No API versioning

All routes are /api/ - breaking changes will require coordinated client updates.

15.3 Scalability Assessment

Current capacity	Single Node.js process + single MongoDB instance. Suitable for ~1,000 concurrent WebSocket connections and ~100 HTTP req/s with caching.
Bottleneck 1	In-memory Socket.IO - single process cannot distribute events across multiple app instances.
Bottleneck 2	In-memory cache - per-process, not shared. Each horizontal replica has independent stale cache.
Bottleneck 3	No admin list pagination - full collection scans on every list request at large data volumes.
Scaling path	Add Socket.IO Redis adapter → enables multi-instance deployment. Replace responseCache with Redis → shared cache. Add pagination → reduces query cost.

15.4 Recommended Improvements (Priority Order)

Priority	Area	Recommendation
1 - URGENT	Fix hardcoded secrets	Set JWT_SECRET and CORS_ORIGIN in .env and add startup validation that throws if they are defaults.
2 - HIGH	Add test suite	Jest + supertest for integration tests on at least the auth, match lifecycle, and score flows.
3 - HIGH	Redis cache + Socket.IO adapter	Enables horizontal scaling and eliminates per-process cache staleness.
4 - HIGH	Rate limiting	express-rate-limit on /api/auth/login and /api/auth/register.
5 - HIGH	Refresh token flow	Short-lived access tokens (15min) + refresh tokens stored in DB/Redis.
6 - MEDIUM	Add pagination	page + limit query params on all admin list endpoints.
7 - MEDIUM	Helmet.js	One-liner: app.use(require('helmet')()) adds 11 security headers.
8 - MEDIUM	Structured logging	Replace console.log with pino or winston. Add request ID via express-request-id.
9 - MEDIUM	TypeScript migration	Gradual migration starting with models and services for compile-time safety.
10 - LOW	Separate scraper service	Move puppeteer/cheerio to a standalone utility container to reduce production image size.
11 - LOW	API versioning	/api/v1/ prefix for all routes enables non-breaking future changes.
12 - LOW	OpenAPI/Swagger spec	Auto-generate from Zod schemas (zod-to-openapi) for client codegen and interactive docs.

15.5 Final Assessment

The Cricbuzz - Backend is a **well-architected, production-class foundation** for a cricket management platform. The codebase demonstrates strong software engineering principles: clean separation of concerns, consistent patterns, data integrity enforcement via state machines, and a thoughtful dual-surface API design (admin + public). The real-time architecture is correctly integrated at the service layer.

The primary gaps are operational rather than architectural: missing test coverage, insecure defaults in configuration, and scalability limits inherent to a single-process in-memory design. These are addressable without significant

refactoring. The modular structure makes it straightforward to add Redis, tests, or TypeScript incrementally.

A new developer with familiarity in Node.js and Express should be productive within one working day using this documentation and the included seed script.

This document was auto-generated from a full static analysis of the project source code. All API routes, database schemas, service logic, middleware, and configuration files have been individually reviewed and documented. Generated: June 2026.